🧠 Image Denoising with Autoencoders

This notebook is part of the **1st Summer School on Deep Learning: Shaping the Future of AI** and demonstrates how unsupervised deep learning can be used to perform **image denoising** using **Autoencoders**.

🧑‍🏫 **Instructor**: Dr. Jignesh S. Bhatt - IIIT Vadodara
📝 **Prepared by**: Vikas Vaghasiya - M.Tech Student

📅 **Date**: May 30, 2025
🕑 **Time**: 2:30 PM – 4:30 PM IST

📌 Objectives

- To Implement a basic autoencoder architecture for denoising
- To Train on image data (MNIST)
- To Visualize compressed (latent) features using t-SNE
- To Perform image reconstruction and denoising by adding noise to images

```
# Stage 1: Import Libraries and Setup
# These tools help us build the Autoencoder, load digit images, and show pictures
import torch  # For neural networks
import torch.nn as nn  # For network layers
import torch.optim as optim  # For training the network
import torchvision  # For the MNIST dataset (digit images)
import torchvision.transforms as transforms  # To turn images into numbers
import matplotlib.pyplot as plt  # To show pictures
import numpy as np  # For math
from sklearn.manifold import TSNE  # To make a 2D map using t-SNE

# Make results consistent every time we run
torch.manual_seed(0)
np.random.seed(0)

# Use GPU if available, else CPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using: {device}")
```

```
Using: cuda
```

```
# Stage 2: Load MNIST Data
# MNIST has 28x28 pictures of handwritten digits (0-9)
print("Loading digit images...")
transform = transforms.ToTensor()  # Turns images into numbers (0 to 1)

# Get training (to learn) and test (to check) data
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, transform=transform, download=True)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, transform=transform, download=True)

# Process data in groups of 100 images
train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batch_size=100, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=100, shuffle=False)
print("Data loaded!")

# Function to add noise (makes images look messy)
def add_noise(images):
    noise = torch.randn_like(images) * 0.5  # Add light "static" to images
    noisy_images = images + noise
    noisy_images = torch.clamp(noisy_images, 0., 1.)  # Keep values between 0 and 1
    return noisy_images
```

```
Loading digit images...
Data loaded!
```

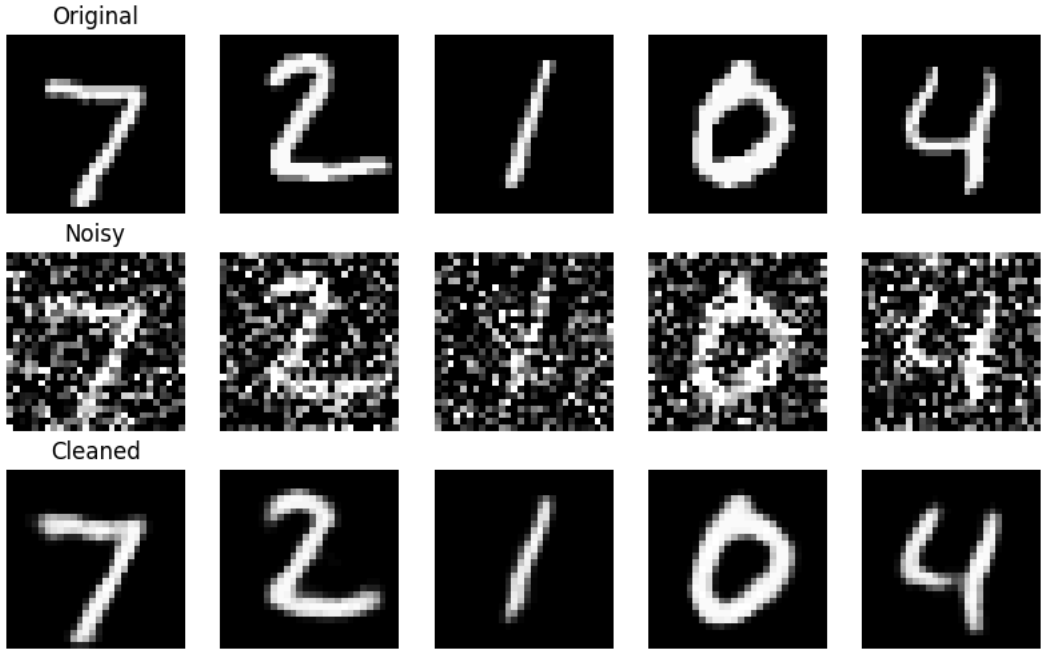
```
# Stage 3: Define a Simple Convolutional Autoencoder
# The Autoencoder learns to clean noisy images by compressing and reconstructing them
class SimpleConvAE(nn.Module):
    def __init__(self):
        super(SimpleConvAE, self).__init__()
        # Encoder: Turns a 28x28 image into a small code
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1),  # [batch, 16, 14, 14]
            nn.ReLU(),  # Add some "energy"
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1),  # [batch, 32, 7, 7]
            nn.ReLU(),
            nn.Flatten(),  # [batch, 32*7*7 = 1568]
            nn.Linear(32*7*7, 128),  # Shrink to 128 numbers (latent code)
```

```
            nn.ReLU()
        )
        # Decoder: Turns the code back into a 28x28 image
        self.decoder = nn.Sequential(
            nn.Linear(128, 32*7*7),  # Expand the code
            nn.Unflatten(1, (32, 7, 7)),  # [batch, 32, 7, 7]
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1, output_padding=1),  # [batch, 16, 14, 14]
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, output_padding=1),  # [batch, 1, 28, 28]
            nn.Sigmoid()  # Make numbers between 0 and 1
        )

    def forward(self, x):
        code = self.encoder(x)  # Compress the image to a code
        reconstructed = self.decoder(code)  # Reconstruct the image from the code
        return reconstructed, code  # Return the cleaned image and the code

# Loss function: Checks how good the Autoencoder's images are
def ae_loss(recon_x, x):
    # Compare reconstructed image to original
    return nn.functional.binary_cross_entropy(recon_x, x, reduction='sum')


# Stage 4: Train the Autoencoder
print("Training the Autoencoder...")
model = SimpleConvAE().to(device)  # Create Autoencoder
optimizer = optim.Adam(model.parameters(), lr=0.001)  # Tool to improve the Autoencoder
num_epochs = 5  # Train for 50 rounds

for epoch in range(num_epochs):
    model.train()  # Training mode
    total_loss = 0
    for data, _ in train_loader:
        data = data.to(device)  # Move to GPU/CPU
        noisy_data = add_noise(data).to(device)  # Make noisy
        optimizer.zero_grad()  # Reset updates
        recon_batch, _ = model(noisy_data)  # Get cleaned images
        loss = ae_loss(recon_batch, data)  # Compare to clean images
        loss.backward()  # Learn from mistakes
        total_loss += loss.item()
        optimizer.step()  # Update the Autoencoder
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {total_loss / len(train_loader.dataset):.2f}")
print("Training finished!")
```

```
Training the Autoencoder...
Epoch 1/5, Loss: 164.72
Epoch 2/5, Loss: 92.15
Epoch 3/5, Loss: 84.38
Epoch 4/5, Loss: 81.38
Epoch 5/5, Loss: 79.77
Training finished!
```

```
# Stage 5: Show Denoising
# Show how the Autoencoder cleans noisy images
print("Showing cleaned images...")
model.eval()
with torch.no_grad():
    data, _ = next(iter(test_loader))  # Get 5 test images
    data = data[:5].to(device)
    noisy_data = add_noise(data).to(device)
    recon, _ = model(noisy_data)  # Clean the noisy images

    plt.figure(figsize=(10, 6))
    for i in range(5):
        plt.subplot(3, 5, i+1)  # Original
        plt.imshow(data[i].cpu().squeeze(), cmap='gray')
        plt.axis('off')
        if i == 0:
            plt.title('Original')
        plt.subplot(3, 5, 5+i+1)  # Noisy
        plt.imshow(noisy_data[i].cpu().squeeze(), cmap='gray')
        plt.axis('off')
        if i == 0:
            plt.title('Noisy')
        plt.subplot(3, 5, 10+i+1)  # Cleaned
        plt.imshow(recon[i].cpu().squeeze(), cmap='gray')
        plt.axis('off')
        if i == 0:
            plt.title('Cleaned')
    plt.show()
print("Look! The Autoencoder makes noisy images clear again!")
```

```
Showing cleaned images...
```

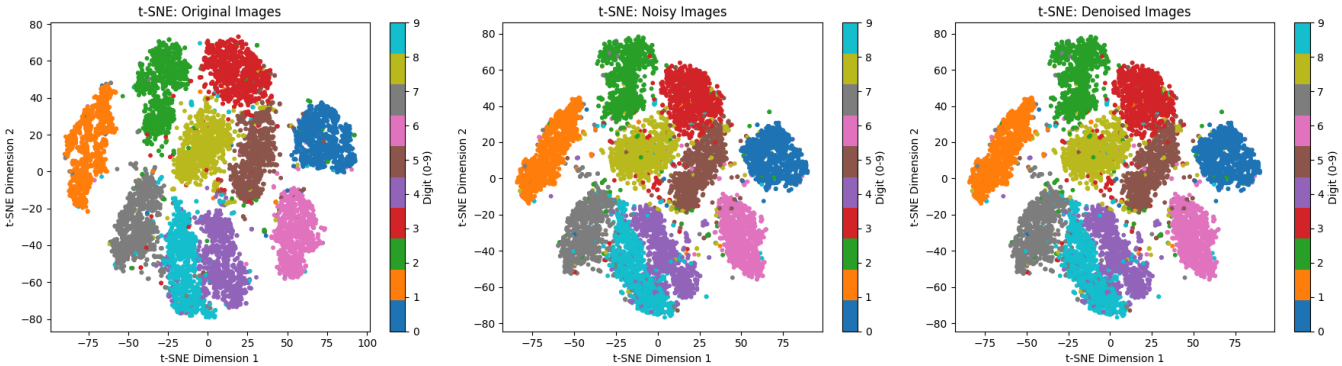



```
Look! The Autoencoder makes noisy images clear again!
```

```
# Stage 6: Show t-SNE Visualizations
# Show how the latent representations of original, noisy, and denoised images are organized
print("Showing t-SNE maps for original, noisy, and denoised images...")
model.eval()
latent_original = []
latent_noisy = []
latent_denoised = []
labels_list = []

with torch.no_grad():
    for data, labels in test_loader:
        data = data.to(device)
        noisy_data = add_noise(data).to(device)

        # Get latent codes for original images
        _, code_original = model(data)
        latent_original.append(code_original.cpu().numpy())

        # Get latent codes for noisy images
        _, code_noisy = model(noisy_data)
        latent_noisy.append(code_noisy.cpu().numpy())

        # Get latent codes for denoised images (reconstructed from noisy)
        recon, code_denoised = model(noisy_data)
        latent_denoised.append(code_denoised.cpu().numpy())

        labels_list.append(labels.numpy())

# Concatenate all latent codes and labels
latent_original = np.concatenate(latent_original, axis=0)  # Shape: [num_samples, 128]
latent_noisy = np.concatenate(latent_noisy, axis=0)
latent_denoised = np.concatenate(latent_denoised, axis=0)
labels = np.concatenate(labels_list, axis=0)

# Use t-SNE to reduce the 128-dimensional latent codes to 2D
tsne = TSNE(n_components=2, random_state=0, perplexity=30)
tsne_original = tsne.fit_transform(latent_original)
tsne_noisy = tsne.fit_transform(latent_noisy)
tsne_denoised = tsne.fit_transform(latent_denoised)

# Plot the three t-SNE visualizations
plt.figure(figsize=(18, 5))

# t-SNE for original images
plt.subplot(1, 3, 1)
plt.scatter(tsne_original[:, 0], tsne_original[:, 1], c=labels, cmap='tab10', s=10)
plt.colorbar(label='Digit (0-9)')
plt.title('t-SNE: Original Images')
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')

# t-SNE for noisy images
```

```
plt.subplot(1, 3, 2)
plt.scatter(tsne_noisy[:, 0], tsne_noisy[:, 1], c=labels, cmap='tab10', s=10)
plt.colorbar(label='Digit (0-9)')
plt.title('t-SNE: Noisy Images')
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')

# t-SNE for denoised images
plt.subplot(1, 3, 3)
plt.scatter(tsne_denoised[:, 0], tsne_denoised[:, 1], c=labels, cmap='tab10', s=10)
plt.colorbar(label='Digit (0-9)')
plt.title('t-SNE: Denoised Images')
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')

plt.tight_layout()
plt.show()
print("These t-SNE maps show how digits are grouped at each stage!")
```

```
Showing t-SNE maps for original, noisy, and denoised images...
```




```
These t-SNE maps show how digits are grouped at each stage!
```

Start coding or generate with AI.